Analisador espectral do Morphe

Como funciona, do programa em Python ao hardware na FPGA

Antonio Nicassio Santos Lima
Estágio no LabPS/UEFS — supervisão: Prof. Armando S. Sanca

14 de outubro de 2026

Resumo

O analisador espectral do Morphe mostra, quadro a quadro, o espectro de um sinal real ligado ao conversor analógico-digital da placa DE1-SoC. Cada quadro atravessa as três camadas do sistema: o programa em Python, no computador, pede a captura e prepara o sinal; o servidor em C, no processador ARM da placa, comanda o hardware; e a FPGA amostra o sinal em instantes fixados por um contador e calcula a FFT. Este documento segue um quadro do começo ao fim, camada por camada, e explica cada conta e cada escala no caminho. Cada afirmação aponta para o arquivo e a linha de onde saiu (branch `estagio/v1.8-codec`); as medidas vêm do diário do estágio.

Sumário

1 Visão geral **2**

2 O programa em Python **4**
2.1 Onde está cada coisa . . . 4
2.2 As opções da janela . . . 4
2.3 O laço de quadros . . . 4
2.4 A captura . . . 5
2.5 O processamento de um quadro . . . 5
2.6 Uma FFT de 1024 pontos na placa . . . 6
2.7 FFT de 4096 e 16 384 pontos . . . 7
2.8 A tela: média, fase, salvar . . . 7

3 O protocolo entre o PC e a placa **8**

4 O servidor em C **8**
4.1 Como o ARM enxerga a FPGA . . . 8
4.2 Proteções antes de tocar na FPGA . . . 9
4.3 O laço principal . . . 9
4.4 OP_ADC: `handle_adc` . . . 9
4.5 OP_FFT: `handle_fft` e `fft_run_block` . . . 10

5 O hardware na FPGA **10**
5.1 As memórias de duas portas . . . 10
5.2 O controlador do ADC: `adc_captura.v` . . . 11
5.3 O wrapper da FFT: `fft_wrapper.v` . . . 12

6 O caminho numérico completo **13**

7 Tempo de um quadro **13**

8 Limites e cuidados **13**

9 Para seguir no código **14**

1 Visão geral

A figura 1 mostra as peças envolvidas. O usuário abre a janela "Analisador espectral (ADC → FFT na FPGA)" na tela inicial do aplicativo (`Python/morphe_app.py:110`), escolhe canal, taxa de amostragem, número de pontos e janela, e clica em *Iniciar*. A partir daí, até clicar em *Parar*, o programa repete o ciclo da figura 2.

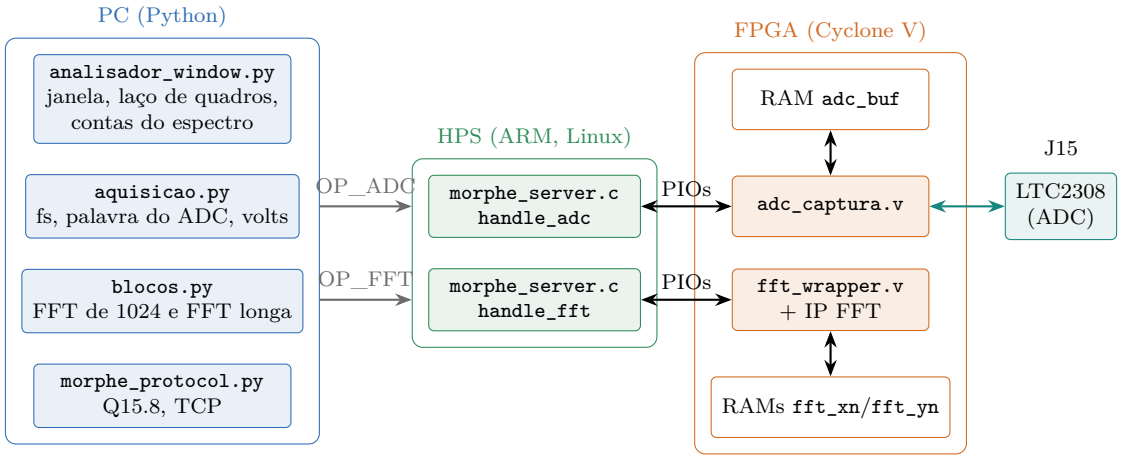


Figura 1: As peças do analisador, nas três camadas. As setas cinzas são pedidos TCP; "PIOs" são os registradores de controle que o ARM escreve e lê pela ponte leve.

Três ideias valem para tudo o que vem depois:

- **Captura e FFT são pedidos separados**, cada um numa conexão TCP própria. O servidor atende uma conexão por vez; entre um quadro e o próximo, outra pessoa pode usar a mesma placa.
- **Quem calcula a FFT é a FPGA**, com o IP FFT da Intel (1024 pontos, ponto flutuante em bloco). O PC só prepara o sinal e interpreta o resultado. O NumPy refaz a mesma conta apenas para mostrar quanto as duas concordam.
- **Acima de 1024 pontos** (4096 e 16 384), o PC divide a FFT em várias FFTs de 1024 na placa e as junta pelo algoritmo de quatro passos. O resultado é exato, sem aproximação.

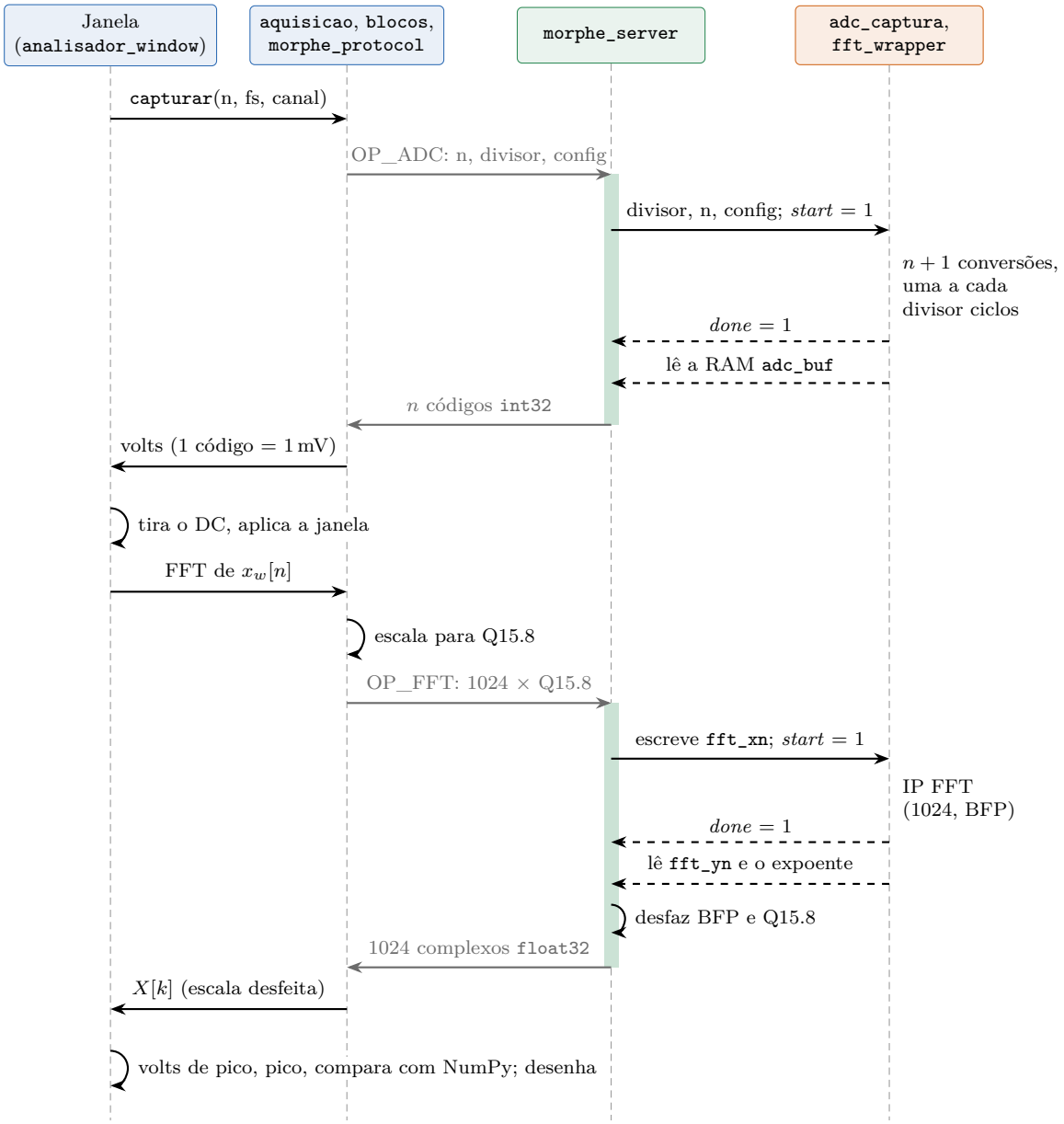


Figura 2: Um quadro do analisador com 1024 pontos. Com 4096 ou 16 384, o trecho da FFT se repete 4 ou 16 vezes (seção 2.7).

2 O programa em Python

2.1 Onde está cada coisa

arquivo	papel no analisador
`analisador_window.py`	a janela: opções, laço de quadros, contas do espectro, gráficos
`aquisicao.py`	captura pelo ADC: palavra de configuração, fs real, volts, saturação
`blocos.py`	`FftPlaca` (uma FFT na placa) e `fft_longa` (quatro passos)
`morphe_protocol.py`	monta e decodifica as mensagens binárias; `TcpClient`
`dsp_core.py`	codificação Q15.8 (`fft_q1508_encode`), `.mrph`
`morphe_config.py`	os mesmos limites de `C/morphe_config.h` (1024, 32 768, 50 MHz)

Tabela 1: Os módulos Python do analisador (pasta `Python/`).

2.2 As opções da janela

Montadas em `AnalisadorWindow._build_sidebar` (`analisador_window.py:225`):

opção	o que faz
Modo / Canal	simples (CH0 a CH7 contra o terra, 0 a 4,095 V) ou diferencial (CH0−CH1 … CH6−CH7, −2,048 a +2,047 V)
fs (Hz)	taxa de amostragem pedida, de 1 kHz a 200 kHz
Pontos	1024 (1 FFT na FPGA), 4096 (4 FFTs) ou 16 384 (16 FFTs)
Janela	Hann, Blackman-Harris, flat-top, retangular
Média	nenhuma, média de potência de 4 ou 16 quadros, retenção de pico
Tirar o nível DC	subtrai a média do quadro antes da FFT
Mostrar a fase	terceiro gráfico, $\angle X(f)$
Stem	desenha as amostras como hastes, sem ligar os pontos

Tabela 2: Opções da janela do analisador.

O texto de informação (`_atualiza_info`, linha 342) mostra a **fs real**, a resolução em frequência f_s/N e a duração do quadro N/f_s, porque a fs pedida raramente é a que o hardware usa (seção 2.4).

2.3 O laço de quadros

`_on_iniciar` (`analisador_window.py:368`) cria o cliente TCP a partir do painel de conexão e dispara uma **thread de trabalho** (`worker`, linha 395), que repete até o evento `parar` ser acionado:

```
cap = aq.capturar(client, n, fs, modo, canal, guardar=False)   # pedido 1: ADC
q = processar_quadro(cap.volts, cap.fs, janela, tirar_dc, fft)  # pedido(s) 2: FFT
with self._trava_q:
    self._novo = q                                              # entrega a tela
```

A thread não desenha: o Tk só pode ser usado pela thread principal. Por isso há dois ritmos independentes:

- a thread produz quadros tão rápido quanto a placa e a rede permitem, e deixa sempre só o **último** em `self._novo`, protegido por uma trava;
- a thread do Tk confere a cada `TELA_MS` = 50 ms (`_tela`, linha 504) se há quadro novo e, se houver, acumula a média e redesenha.

Se a tela for mais lenta que a placa, quadros intermediários deixam de ser desenhados, mas a análise não atrasa. As opções que a thread lê (janela, tirar DC) ficam no dicionário `self._cfg`, copiado pela thread do Tk em `_zera_media` (linha 359): variáveis do Tk não podem ser lidas de outra thread.

2.4 A captura

`aquisicao.capturar` (`aquisicao.py:139`) faz três contas antes de qualquer byte sair do PC.

A fs que o hardware realmente usa. A FPGA conta ciclos de um relógio de 50 MHz, e o período de amostragem tem de ser um número inteiro de ciclos, o *divisor*:

$$\text{divisor} = \text{round}\left(\frac{50\,\text{MHz}}{f_s}\right) \in [250,\ 50\,000], \qquad f_{s,\text{real}} = \frac{50\,\text{MHz}}{\text{divisor}}.$$

Pedir 44 100 Hz dá divisor 1134 e $f_{s,\text{real}} = 44\,091{,}7$ Hz (`divisor_para` e `fs_de`, linhas 86 e 93). **Todas as contas seguintes usam a fs real**; é ela que aparece no eixo de frequência.

A palavra de configuração do LTC2308. `palavra_config` (linha 63) monta os 6 bits `S/D O/S S1 S0 UNI SLP`, que escolhem o canal, o modo simples ou diferencial e a saída unipolar ou bipolar. No modo simples, o canal é codificado como $\{S1, S0, O/S\}$ (tabela 1 da folha de dados). `SLP` fica sempre em 0: dormir desliga a referência interna, que leva 200 ms para voltar.

O tempo de espera. A captura leva $(n+1) \cdot \text{divisor}/50\,\text{MHz}$ segundos (o primeiro quadro é descartado, seção 5.2). O cliente usa um timeout de pelo menos essa duração mais 10 s: 32 768 amostras a 1 kHz levam 33 s, e o padrão do painel TCP é 10 s.

A resposta são **códigos inteiros** do conversor: 0 a 4095 no modo simples e −2048 a 2047 no diferencial (o servidor já estendeu o sinal). Com a referência interna de 4,096 V, **1 código = 1 mV**, e a conversão para volts é `codigos * 0.001`. O cliente confere que voltaram exatamente n amostras e que o divisor usado (campo `extra` da resposta) é o pedido. `saturou` (linha 357) avisa se algum código bateu no limite da faixa.

2.5 O processamento de um quadro

`processar_quadro` (`analisador_window.py:120`) é uma função pura, sem Tk: recebe os volts e uma função `fft`, e devolve um dicionário com tudo o que a tela mostra. Os testes de bancada a chamam com `np.fft.fft` no lugar da placa.

Passo 1: nível DC e janela.

$$\bar{x} = \frac{1}{N}\sum_n x[n], \qquad x_w[n] = (x[n] - \bar{x})\, w[n].$$

No modo simples o sinal nunca é negativo: um gerador com offset de 2 V produz uma raia enorme em $f = 0$ que, vazando pela janela, esconderia os tons baixos. Subtrair a média remove essa raia.

As janelas (`janela`, linha 109) são **periódicas**, de soma conhecida: Hann, Blackman-Harris de 4 termos, flat-top de 5 termos e retangular. A de Hann, por exemplo, é `np.hanning(n+1)` sem a última amostra. A escolha é um compromisso (tabela 3).

janela	erro máximo de amplitude	uso
Retangular	até 3,9 dB a menos	só com número inteiro de períodos no quadro
Hann	até 1,4 dB	uso geral
Blackman-Harris	até 0,8 dB; lóbulos abaixo de −92 dB	separar tons fracos de fortes
Flat-top	menos de 0,02 dB	**medir amplitude**; pico mais largo

Tabela 3: Erro de amplitude de um tom que cai entre dois bins, por janela (comentário de `analisador_window.py:48`, conferido em bancada em 30/09).

Passo 2: a FFT, na FPGA. $X = \texttt{fft}(x_w)$, com o tempo medido. A função `fft` é montada em `_on_iniciar` (linha 389):

```
fft_placa = blocos.FftPlaca(client, normalizar=True)
def fft(x):
    return blocos.fft_longa(x, fft_placa) if x.size > 1024 else fft_placa(x)
```

Os detalhes estão nas seções 2.6 e 2.7.

Passo 3: de $|X[k]|$ para volts de pico. Uma senoide de amplitude A, janelada por w, produz no seu bin $|X[k]| = A \cdot \frac{1}{2}\sum_n w[n]$. Então

$$\hat{A}[k] = \frac{|X[k]|}{\frac{1}{2}\sum_n w[n]}, \qquad \hat{A}[0] = \frac{|X[0]|}{\sum_n w[n]},$$

porque no DC o ganho é a soma inteira, e não a metade. O espectro mostrado é **completo, de 0 a f_s** (os N bins), com uma linha pontilhada em $f_s/2$: como o sinal é real, $X[N-k] = X^*[k]$, e um tom em f aparece também em $f_s - f$, com a mesma amplitude. O eixo vertical é **linear, em volts de pico**, conforme decidido em 01/10 (`docs/DECISOES.md`).

Passo 4: o pico. É procurado só na metade de baixo, ignorando os três primeiros bins (`BINS_DC`, o que sobra do DC depois da janela). A frequência é refinada por uma **parábola** nos três bins em volta do máximo, em dB (linha 156). Com a, b, c os valores em dB dos bins $k-1$, k e $k+1$:

$$\delta = \frac{1}{2}\,\frac{a-c}{a-2b+c}, \qquad f_{\text{pico}} = (k+\delta)\,\frac{f_s}{N}, \qquad |\delta| \le 0{,}5.$$

O lóbulo de uma janela é quase parabólico em escala logarítmica, e isso corrige o erro de até meio bin do índice inteiro. O limite em ±0,5 bin existe porque, perto do DC ou só com ruído, a parábola pode apontar longe.

Passo 5: a prova. O NumPy calcula `np.fft.fft` do **mesmo** x_w enviado à placa, e a tela mostra

$$\text{SNR} = 20\log_{10}\frac{\|X_{\text{NumPy}}\|}{\|X_{\text{FPGA}} - X_{\text{NumPy}}\|} \quad \text{(dB)}$$

como "FPGA × NumPy". Medido na placa 1 em 30/09/2026: **97,6 dB** em todos os quadros com 1024 pontos, e 97,4 a 97,8 dB com 4096. É a confirmação, a cada quadro, de que o espectro saiu mesmo da FPGA e com que precisão.

2.6 Uma FFT de 1024 pontos na placa

`FftPlaca` (`blocos.py:106`):

```
x = dsp.pad_zeros_to(x, 1024)
req, escala = build_fft_request(x, normalizar=True)
return decode_fft_response(self.client.request(req), escala)
```

O hardware recebe números em **Q15.8**: inteiros de 24 bits com sinal, dos quais 8 são fracionários. A faixa é ±32 768 e o passo, 1/256. Um sinal do ADC tem no máximo ±2 V; enviado como está, usaria uns 10 dos 24 bits, e o erro de arredondamento seria grande em relação ao sinal.

Por isso a **normalização** (`escala_para_q1508`, `morphe_protocol.py:198`):

$$s = \frac{0{,}98 \cdot 32\,767{,}996}{\max_n |x_w[n]|}, \qquad q[n] = \mathrm{round}\big(256 \cdot s\, x_w[n]\big),$$

com q saturado em 24 bits pela função `fft_q1508_encode`. Na volta, `decode_fft_response` (linha 450) divide: $X = X_{\text{placa}}/s$. A conta é exata, porque a DFT é linear: $\mathrm{DFT}(s\,x)/s = \mathrm{DFT}(x)$. Muda só o erro de quantização, que cai 6,02 dB por bit de faixa recuperado, até o teto de cerca de 92 dB do IP. Cada bloco é normalizado por conta própria. As medidas estão na seção 3 de `docs/PRECISAO-NUMERICA.md`.

Cada amostra Q15.8 viaja num `int32` big-endian, com os 8 bits de cima repetindo o bit de sinal; o hardware só usa os 24 de baixo.

2.7 FFT de 4096 e 16 384 pontos

`fft_longa` (`blocos.py:241`) calcula uma DFT de $N = 1024\,M$ pontos pelo algoritmo de **quatro passos** (Cooley-Tukey de raiz M). Escrevendo $n = n_1 + Mn_2$ e $k = k_2 + 1024\,k_1$, com $0 \le n_1, k_1 < M$ e $0 \le n_2, k_2 < 1024$:

$$X[k_2 + 1024\,k_1] = \sum_{n_1=0}^{M-1} \underbrace{e^{-j2\pi n_1 k_1/M}}_{\text{passo 4: DFT de } M}\ \underbrace{e^{-j2\pi n_1 k_2/N}}_{\text{passo 3: giro}}\ \underbrace{\sum_{n_2=0}^{1023} x[n_1 + Mn_2]\, e^{-j2\pi n_2 k_2/1024}}_{\text{passos 1 e 2: } A[n_1][k_2]\text{, na placa}}.$$

1. separa x em M subsequências decimadas, $x_{n_1}[n_2] = x[n_1 + Mn_2]$;
2. calcula a FFT de 1024 de cada uma **na placa**: M pedidos OP_FFT;
3. multiplica pelos fatores de giro $e^{-j2\pi n_1 k_2/N}$;
4. calcula, no PC, 1024 DFTs de M pontos ao longo de n_1.

No código, os passos 3 e 4 são duas linhas:

```
giro = np.exp(-2j*np.pi * np.outer(np.arange(M), np.arange(1024)) / N)
return np.fft.fft(A * giro, axis=0).reshape(-1)
```

O trabalho pesado (M FFTs de 1024) fica com a FPGA; o PC faz só $1024\,M$ multiplicações complexas e as DFTs minúsculas de M pontos. Com 16 384 pontos são 16 pedidos à placa por quadro, o que reduz a taxa de quadros.

2.8 A tela: média, fase, salvar

- **Média** (`_acumula`, linha 488): sempre sobre a **potência** $\hat{A}^2$, nunca sobre a amplitude ou o complexo. Média de K quadros: $\sqrt{\text{média}(\hat{A}^2)}$ dos últimos K; retenção de pico: $\sqrt{\max \hat{A}^2}$ desde o começo. Mudar a janela, o DC ou o tipo de média zera o acumulador.
- **Fase** (`fase_visivel`, linha 99): $\angle X[k]$ só nos bins com amplitude de pelo menos 2 % da maior. Nos outros, $X[k]$ é ruído e a fase é um número sorteado entre $-\pi$ e π; por isso aparecem como lacuna. A fase é sempre a do quadro atual: cada captura começa num ponto diferente da onda, e uma média de fase não teria sentido.

- **Medidas do quadro**: pico (Hz e mV), nível DC, tempo de captura, tempo da FFT, quadros por segundo, FPGA × NumPy e aviso de saturação.
- **Salvar quadro** (`_on_salvar`, linha 561): grava em `.mrph` o x_w (exatamente o que foi para a FPGA) e o X da placa, no formato que o Comparador abre.

3 O protocolo entre o PC e a placa

Definido em dois arquivos que espelham um ao outro, `Python/morphe_protocol.py` e `C/morphe_protocol.h`. TCP na porta 5000, **uma conexão por pedido**, todos os inteiros em big-endian.

campo	tipo	valor
magic	`u32`	`0x4D52504D` ("MRPM")
version	`u16`	1
opcode	`u16`	2 = FFT, 7 = ADC (há também CONV, FIR, IFFT, IIR, PING, ADC contínuo)
dtype	`u16`	1 = `int32`
flags	`u16`	ADC: palavra de 6 bits do LTC2308; FFT: 0
`n_x`	`u32`	ADC: número de amostras; FFT: 1024
`n_h`	`u32`	ADC: divisor; FFT: 0

Tabela 4: Cabeçalho de pedido, 20 bytes (`struct.pack(»IHHHHII", ...)`).

O cabeçalho da resposta também tem 20 bytes: magic `0x4D52504E` ("MRPN"), version, opcode (espelhado), dtype, status (0 = OK), `n_out` e `extra`.

	pedido	resposta
OP_ADC (7)	só o cabeçalho	n códigos `int32`; `extra` = divisor usado
OP_FFT (2)	cabeçalho + 1024 `int32` Q15.8 (4 KiB)	1024 complexos `float32`, re e im intercalados (8 KiB)

Tabela 5: Os dois pedidos do analisador.

Em erro, o status é diferente de zero e a carga é a mensagem em texto (`n_out` = número de bytes). O Python a transforma em `RuntimeError("erro do servidor: ...")`, que a janela mostra numa caixa de diálogo. Por exemplo, com a FPGA ainda no bitstream de fábrica, a resposta é o status 8, "FFT: FPGA nao preparada – rode ./morphe-up.sh nesta placa".

`TcpClient.request` (`morphe_protocol.py:370`) abre a conexão, desliga o algoritmo de Nagle (`TCP_NODELAY`, que atrasaria mensagens curtas), envia o pedido, lê exatamente os 20 bytes do cabeçalho, calcula o tamanho da carga pelo opcode (FFT: $8\,n_{\text{out}}$; ADC: $4\,n_{\text{out}}$) e lê exatamente esse tanto.

4 O servidor em C

`C/morphe_server.c` é um único processo, no Linux do ARM da placa, executado como root por causa do `/dev/mem`.

4.1 Como o ARM enxerga a FPGA

O ARM e a FPGA estão no mesmo chip (Cyclone V SoC), ligados por pontes AXI. Do lado do ARM, cada memória e cada registrador da FPGA é um **endereço físico**. `fpga_init` (linha 413) abre `/dev/mem` e mapeia duas janelas no espaço do processo com `mmap` (tabela 6).

janela	base física	o que há dentro (offsets de `C/hps_0.h`)
ponte HPS→FPGA	`0xC0000000`	RAMs: `fft_yn_re` 0x18000, `fft_yn_imag` 0x19000, `fft_xn_re` 0x1A000, `fft_xn_imag` 0x1B000 (4 KiB cada, 1024 `int32`); `adc_buf` 0x20000 (128 KiB, 32 768 `int32`)
ponte leve	`0xFF200000`	PIOs da FFT: `bfp_exponent` 0x50, `done` 0x60, `inverse` 0x70, `start` 0x80. PIOs do ADC: `start` 0xD0, `done` 0xE0, `config` 0xF0, `divisor` 0x100, `namostras` 0x110, `contador` 0x120

Tabela 6: As duas janelas de memória que o servidor mapeia.

Depois do mapeamento, escrever num registrador da FPGA é uma atribuição de ponteiro: `*g_pio_adc_divisor = divisor;`. Os ponteiros são `volatile`, para o compilador não eliminar nem reordenar os acessos. Os endereços de `hps_0.h` saem do projeto do Platform Designer (`Quartus/gen_hps_header.py`): **se o bitstream muda de mapa, o servidor tem de ser recompilado com o `hps_0.h` novo.**

4.2 Proteções antes de tocar na FPGA

Acessar pela ponte um endereço que a FPGA não implementa **trava o barramento do ARM**, sem timeout: a placa some da rede. Isso aconteceu em 21/09, quando o servidor subia no boot com a FPGA ainda no bitstream de fábrica. Daí duas proteções:

- `fpga_preparada` (linha 495): só deixa uma operação seguir se existe o arquivo `/var/run/morphe-fpga-preparada`, que o `morphe-up.sh` cria depois de programar o `.sof` do Morphe. Na primeira vez, zera os PIOs de *start*.
- `adc_recusa` (linha 1097), só para o ADC: compara o *timestamp* do `sysid` gravado no bitstream com o `SYSID_QSYS_TIMESTAMP` com que o servidor foi compilado. Se forem diferentes, o bitstream é outro e os PIOs do ADC podem nem existir. Confere também o divisor (250 a 50 000), o dtype e o bit SLP.

4.3 O laço principal

`main` (linha 1566) abre o socket na porta 5000 e repete `accept` → `serve_connection` → `close`. **Uma conexão por vez**: enquanto uma captura ou FFT roda, a próxima conexão espera na fila do `listen`. É isso que permite a várias pessoas usarem a mesma placa: cada quadro do analisador são duas ou mais conexões curtas, e as dos outros entram entre elas.

`serve_connection` (linha 1524) lê o cabeçalho, confere magic e versão e despacha pelo opcode. Em paralelo, uma thread responde na porta UDP 5001 quem está usando a placa e qual operação está rodando (`thread_estado`); é o que a tela de descoberta mostra como "ocupada".

4.4 OP_ADC: `handle_adc`

Linha 1138, resumida (comentários sem acento, como no código):

```
*g_pio_adc_divisor   = divisor;      /* periodo em ciclos de 50 MHz */
*g_pio_adc_namostras = n;
*g_pio_adc_config    = cfg;          /* bit 6 = 0: captura unica */
*g_pio_adc_start = 0; usleep(1); *g_pio_adc_start = 1;  /* borda de subida */

wait_done(g_pio_adc_done, duracao_ms + 2000);           /* polling do done */
*g_pio_adc_start = 0;                                    /* libera o hardware */

for (i = 0; i < n; i++)                                  /* le a RAM pela ponte */
    tx[i] = adc_codigo(g_adc_buf[i], bipolar);           /* 12 bits -> int32 */
```

- O **aperto de mão** *start*/*done* é o mesmo em todos os blocos do Morphe: o ARM sobe *start*; o hardware trabalha, sobe *done* e o mantém em 1 até o ARM baixar *start*. Assim nenhum dos dois lados perde o evento do outro.
- `wait_done` (linha 529) lê o PIO a cada 50 μs, com timeout.
- `adc_codigo` (linha 1132) pega os 12 bits de baixo da palavra e, no modo bipolar (`UNI` = 0), estende o sinal do complemento de dois.
- A resposta leva o **divisor no campo `extra`**, para o cliente confirmar a fs.

4.5 OP_FFT: `handle_fft` e `fft_run_block`

Linhas 1375 e 1334, resumidas:

```
recv_exact(sock, rx_buf, 1024*4);              /* 1024 int32 Q15.8 */
decode_samples_to_q1508(rx_buf, 1024, x);      /* big-endian; satura em 24 bits */
for (i = 0; i < 1024; i++) {
    g_fft_xn_re[i] = x[i]; g_fft_xn_imag[i] = 0;   /* entrada real */
}
*g_pio_fft_inverse = 0;                        /* ANTES do start */
*g_pio_fft_start = 0; usleep(1); *g_pio_fft_start = 1;
wait_done(g_pio_fft_done, 5000);
*g_pio_fft_start = 0;

exp = *g_pio_fft_bfp_exponent & 0x3F;          /* 6 bits, com sinal */
for (k = 0; k < 1024; k++) {
    re[k] = g_fft_yn_re[k]; im[k] = g_fft_yn_imag[k];
}
escala = ldexpf(1.0f, -exp) / 256;             /* desfaz BFP e Q15.8 */
/* envia (float) re[k] * escala e (float) im[k] * escala, big-endian */
```

O ponto central é o **expoente de ponto flutuante em bloco** (BFP). Para não estourar os 24 bits ao longo dos estágios, o IP divide os dados por 2 sempre que um estágio cresceria demais, e conta quantas vezes fez isso. Esse número sai junto com o resultado: um único expoente para o bloco inteiro de 1024 bins. O servidor o lê do PIO `fft_bfp_exponent`, multiplica a saída por $2^{-\mathrm{exp}}$ para recuperar a escala verdadeira e divide por 256 para tirar o Q15.8 da entrada. O que chega ao Python é a DFT de $s\,x_w$ em ponto flutuante; o Python divide por s (seção 2.6).

O bit `inverse` é escrito *antes* de subir o *start* porque o wrapper o trava na borda de subida (seção 5.3). Cada FFT também grava um arquivo `.mrph` de depuração no cartão da placa (`save_debug_bundle_fft`); os mais antigos são apagados (`MORPHE_DUMP_MAX`, padrão 100).

5 O hardware na FPGA

O projeto Quartus está em `Quartus/`. O topo, `ghrd_top.v`, instancia o sistema do Platform Designer (`soc_system`: HPS, pontes, RAMs *on-chip*, PIOs) e, ao lado dele, os blocos próprios do Morphe. Todos usam o relógio `CLOCK_50`, de 50 MHz.

5.1 As memórias de duas portas

Cada RAM do caminho (`fft_xn_re`, `fft_xn_imag`, `fft_yn_re`, `fft_yn_imag`, `adc_buf`) tem **duas portas**:

- a **porta A**, no barramento, que o ARM lê e escreve pela ponte;
- a **porta B**, exportada para o `ghrd_top.v` e ligada diretamente ao bloco de hardware.

É por elas que os dados passam entre o software e o hardware, sem DMA: o ARM escreve a entrada pela porta A, o hardware a lê pela porta B, escreve o resultado pela porta B de outra RAM, e o ARM o lê pela porta A.

5.2 O controlador do ADC: `adc_captura.v`

Instanciado em `ghrd_top.v:604`, com os pinos `ADC_CONVST`, `ADC_SCLK`, `ADC_DIN` e `ADC_DOUT` ligados ao LTC2308 e a porta B da RAM `adc_buf` (2^{15} palavras).

Por que um controlador próprio. O IP do University Program converte sem parar, no ritmo dele, e entrega só o último valor de cada canal. Ler esse valor a cada $1/f_s$ deixa o instante da amostra incerto em alguns microssegundos, e essa incerteza vira ruído que cresce com a frequência do sinal (`docs/ADC.md`). Aqui, **cada conversão começa exatamente no ciclo em que o contador de período volta a zero**: o instante de amostragem é fixado pelo hardware, com a incerteza de um ciclo de 50 MHz (20 ns). Para um analisador espectral isso é essencial, porque a DFT supõe amostras igualmente espaçadas.

A máquina de estados.

`S_IDLE` na borda de subida de *start*, trava `divisor` (no mínimo 250), `n_amostras`, `config_adc` e `continuo`, e zera o índice e o contador;

`S_PREP` espera 16 ciclos, para garantir o tempo de aquisição do conversor;

`S_CAPT` um **quadro por amostra**, contado pela variável `fase`, de 0 a divisor − 1 (figura 3);

`S_DONE` sobe *done* e espera o ARM baixar *start*.

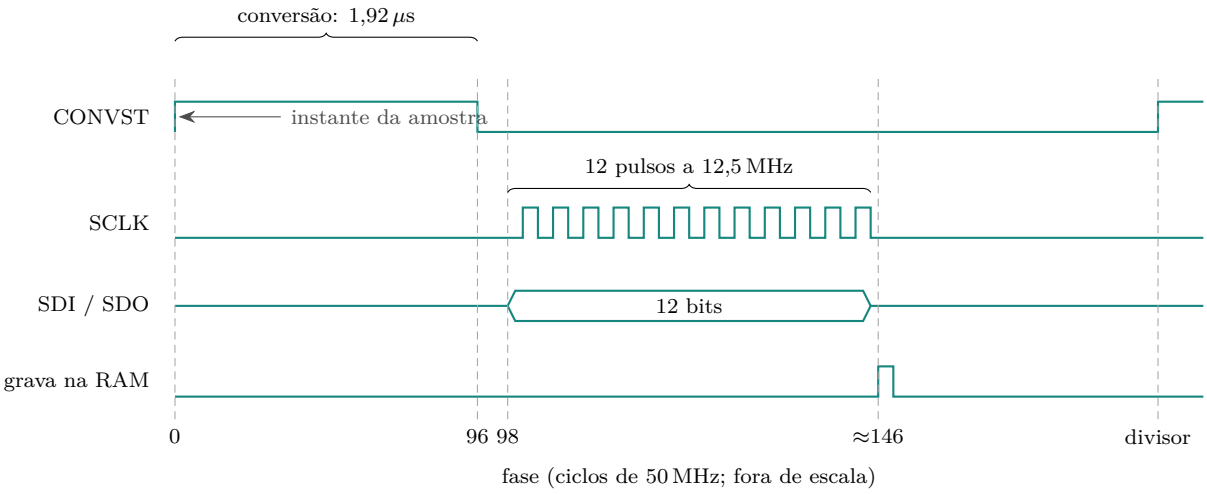


Figura 3: Um quadro de `adc_captura.v` (estado `S_CAPT`). Na fase 0, CONVST sobe e a conversão começa (é o instante da amostra); na fase 96, desce; da fase 98 em diante, 12 pulsos de SCLK leem o resultado pelo SDO (B11 a B0) e, nos 6 primeiros, enviam a configuração pelo SDI; no fim da leitura, a amostra vai para a RAM. Na fase divisor − 1 o contador volta a 0 e o próximo quadro começa. Com divisor 250 (200 kHz), o quadro de cerca de 146 ciclos cabe com folga.

O primeiro quadro é descartado. No LTC2308, a palavra de configuração enviada durante uma leitura vale para a conversão **seguinte**. A conversão do quadro 0 usou a configuração anterior (talvez de outro canal); por isso são feitas $n+1$ conversões, e a primeira não é gravada (registrador `descartar`).

A amostra vai para a RAM crua, nos 12 bits de baixo de uma palavra de 32. O sinal do modo bipolar é estendido pelo servidor, não aqui.

Modo contínuo. Com o bit 6 de `adc_config` em 1 (usado pela janela de Aquisição, não pelo analisador), a RAM vira um buffer circular, e o `contador` diz quantas amostras já foram gravadas, para o servidor copiar as novas enquanto a captura segue.

5.3 O wrapper da FFT: `fft_wrapper.v`

Instanciado em `ghrd_top.v:700`, liga as quatro RAMs (portas B) ao **IP FFT da Intel** (`fft_core`, gerado no Platform Designer a partir de `Quartus/fft_core.qsys`). A configuração do IP: 1024 pontos, arquitetura *Buffered Burst*, entrada de 24 bits, fatores de giro de 18 bits, representação *Block Floating Point*, ordem natural na entrada e na saída, bidirecional (o mesmo IP faz a IFFT com `inverse` = 1).

O IP usa a interface **Avalon-ST**: um fluxo com `valid` e `ready` dos dois lados e marcas de início (`sop`) e fim (`eop`) de pacote. O wrapper converte "uma RAM endereçável" em "um fluxo de 1024 amostras" e o contrário, com a máquina de estados da figura 4.

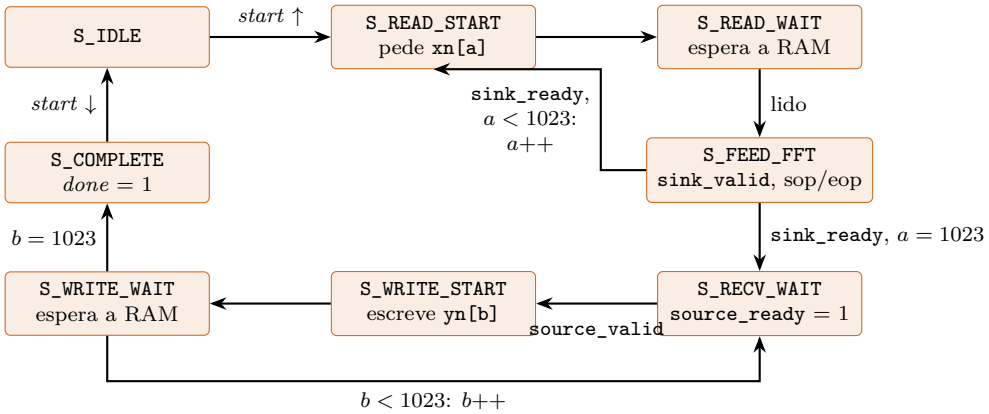


Figura 4: Máquina de estados de `fft_wrapper.v`. A metade de cima alimenta o IP com as 1024 amostras (a = `feed_addr`); a de baixo recebe os 1024 bins e os grava (b = `recv_addr`). No primeiro bin (`source_sop`), o expoente BFP é guardado em `fft_exponent`.

Três detalhes custaram horas de depuração e estão no código por isso:

- **`source_ready` é um fio combinacional** (`state == S_RECV_WAIT`), e não um registrador. Com o *ready* oscilando no meio do quadro, a IFFT perdia o fim do pacote, o wrapper ficava preso em `S_RECV_WAIT` e toda FFT seguinte dava timeout. A versão correta é a do bitstream de 08/09/2026, versionada só em 15/09.
- **`inverse` é travado na borda de subida do *start*.** Por isso o servidor escreve esse PIO antes de subir o *start*.
- **O IP recebe só os 24 bits de baixo** de cada palavra de entrada, e a saída de 24 bits é estendida para 32 antes de ir à RAM. Por isso o cliente codifica em Q15.8 com o sinal estendido.

6 O caminho numérico completo

Juntando as camadas, um bin $X[k]$ passa pelas escalas da figura 5.

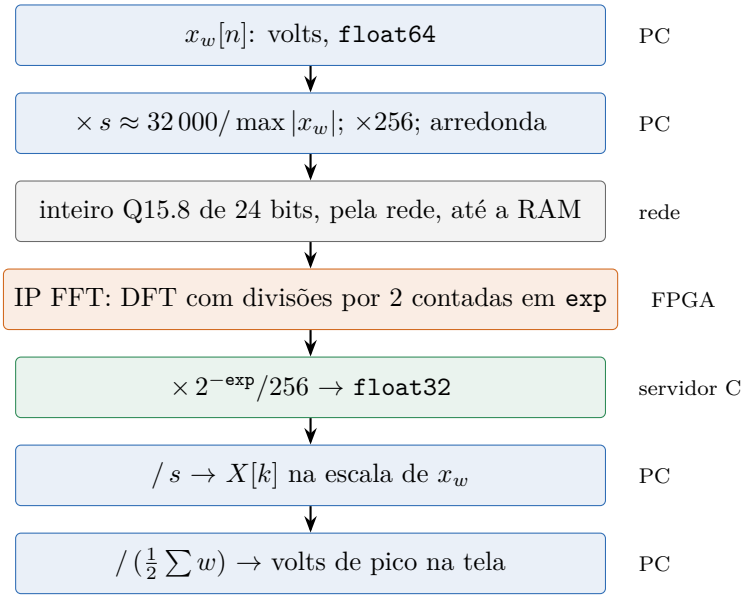


Figura 5: As escalas de um bin, da janela até a tela. Cada multiplicação feita na ida é desfeita na volta; o único efeito líquido é o erro de quantização.

O ADC dá cerca de 9 bits efetivos (ENOB 9,03 e SINAD 56,1 dB, medidos em 07/10/2026 com senoide de gerador), e a FFT da placa concorda com o NumPy a cerca de 97 dB. Portanto **o limite do analisador é o conversor, não a FFT**: o erro da FPGA fica uns 40 dB abaixo do ruído do próprio sinal.

7 Tempo de um quadro

Medido na placa 1, a partir do PC Windows, CH0, 10 kHz, 1024 pontos (30/09/2026):

etapa	tempo
captura no ADC	1025/10 000 Hz ≈ 103 ms, mais a rede
FFT (codificação, rede, cálculo, resposta)	30 a 40 ms
resultado	cerca de 5,6 quadros por segundo

Tabela 7: Tempo de um quadro com 1024 pontos a 10 kHz.

O pedido de FFT sozinho, com a placa livre, mede de 21 a 25 ms (testes do `morphe-up.sh` no diário). O log do servidor separa as partes de cada pedido (`log_tempos`: `entrada`, `fpga`, `saida`), mas o tempo só do hardware ainda não foi registrado no diário. A captura, ao contrário, é limitada pela física: N amostras a f_s levam N/f_s segundos. Com 16 384 pontos a 1 kHz, um quadro leva 16 s só para ser capturado.

8 Limites e cuidados

- **Não há filtro antialiasing** entre o conector J15 e o LTC2308. Qualquer componente acima de $f_s/2$ volta como alias, e a tela não tem como distingui-lo. A fs deve ficar bem acima do dobro da maior frequência presente.

- **Faixa de entrada**: 0 a 4,096 V por pino, em qualquer modo. Tensão negativa pode danificar o conversor. Com gerador de funções: offset de cerca de 2 V e no máximo 4 Vpp, conferidos no osciloscópio antes de ligar; terra no pino 10 do J15, CH0 no pino 2.
- **Tamanho**: até 32 768 amostras por captura única (a RAM `adc_buf`); o analisador usa até 16 384.
- **fs**: de 1 kHz (divisor 50 000) a 200 kHz (divisor 250), sempre 50 MHz dividido por um inteiro.
- **Entrada sem nada ligado** dá um sinal que começa em cerca de 2,4 V e decai nas primeiras amostras (entrada de alta impedância; diário de 30/09). Não é defeito do analisador.
- **Placa compartilhada**: com várias pessoas na mesma placa, os pedidos se intercalam, e a taxa de quadros de cada uma cai.

9 Para seguir no código

quero entender...	começar por
o laço da janela	`Python/analisador_window.py:395`
as contas do espectro	`Python/analisador_window.py:120`
a fs real e a palavra do ADC	`Python/aquisicao.py:63` e `:86`
a normalização Q15.8	`Python/morphe_protocol.py:198` e `:213`
a FFT longa	`Python/blocos.py:241`
o despacho no servidor	`C/morphe_server.c:1524`
a captura no servidor	`C/morphe_server.c:1138`
a FFT no servidor e o BFP	`C/morphe_server.c:1334`
o instante de amostragem	`Quartus/adc_captura.v`, estado `S_CAPT`
a ponte RAM ↔ IP FFT	`Quartus/fft_wrapper.v`
os endereços	`C/hps_0.h` e `C/address_map_arm.h`
precisão numérica	`docs/PRECISAO-NUMERICA.md`
o ADC em detalhe	`docs/ADC.md`

Tabela 8: Pontos de entrada no código.